

Projectile Motion with Air Resistance

Ivan Pestana

August 22, 2026

1 Physical Model

1.1 Motivation

When we consider a projectile launched from ground level with an initial velocity v_0 at an angle θ above the horizontal, the motion can be described by Newton's second law. If we ask the question of which angle provides the maximum range, in the absence of air resistance, the answer is famously 45° . However this is not the case if we include air resistance.

1.2 Newton's Second Law

Newton's second law for the projectile reads

$$\mathbf{F}_r + \mathbf{F}_g = m\mathbf{a}, \quad (1)$$

where $\mathbf{F}_g = (0, -mg)$ is the force of gravity. Writing this equation component by component,

$$a_x = \frac{F_{r,x}}{m}, \quad a_y = -g - \frac{F_{r,y}}{m}, \quad (2)$$

where $F_{r,x}$ and $F_{r,y}$ are still unknown, since we have not yet written $\mathbf{F}_r$ explicitly.

1.3 The Drag Force

When air resistance is included, the projectile is subject to a drag force $\mathbf{F}_r$ that always acts in the opposite direction of the velocity $\mathbf{v}$. Throughout this document, bold symbols denote vectors (e.g. $\mathbf{v}$), while the same symbol in italics denotes its magnitude (e.g. $v = |\mathbf{v}|$).

The form of the drag force depends on the Reynolds number,

$$\text{Re} = \frac{Dv\rho}{\eta}, \quad (3)$$

where D is the projectile's size, ρ the density of air, and η its viscosity.

Re is the ratio of inertial to viscous forces in the surrounding flow. If we have a low Re, viscous forces dominate and the drag force is linear,

$$F_r \propto v \quad (\text{linear, or Stokes, drag}), \quad (4)$$

which is a regime typical of small objects moving slowly through a viscous fluid, such as a droplet. This regime holds for $\text{Re} \lesssim 1$; for example, a $10 \mu\text{m}$ fog droplet settling in air at roughly 2.7 cm/s has $\text{Re} \approx 0.016$ [1]. Turning the proportionality into an equality introduces a constant b , the linear drag coefficient, which for a sphere is given by Stokes' law as $b = 3\pi\eta D$. Since the force points opposite to the velocity,

$$\mathbf{F}_r = -b\mathbf{v}. \quad (5)$$

On the other hand, if we have a high Re , inertial forces dominate and the drag force is quadratic,

$$F_r \propto v^2 \quad (\text{quadratic, or Newtonian, drag}), \quad (6)$$

which is a regime typical of bigger objects moving through air, such as a thrown ball or a fired projectile. This regime holds for speeds between approximately 0.25 m/s and 53 m/s, corresponding to $10^3 < \text{Re} < 2 \times 10^5$ [2].

Between these two regimes, roughly $1 \lesssim \text{Re} \lesssim 10^3$, there is a middle zone where neither law works well, and the drag force has no simple formula, instead being found from experiments or lookup tables [1]. This middle zone does not matter here, since projectile motion falls well inside the quadratic regime.

So far we only know that the drag force grows with the square of the speed – we do not yet have an exact formula. To turn this proportionality into a real equation, we introduce a constant, C , called the drag coefficient:

$$F_r = C v^2. \quad (7)$$

This constant packages together everything else that affects the force: the projectile's shape, its cross-sectional area, and the density of the air. In reality, C can itself change slightly with the speed of the projectile, $C = C(\mathbf{v})$. However, this variation is small within the range of speeds we consider, so throughout this document we treat C as a fixed, known constant. This is an approximation, but a reasonable one: it greatly simplifies the equations while still capturing the essential physics of drag.

We still need the force as a vector, not just a size. To point it in the right direction, we multiply by a unit vector – a vector of length one – that points opposite to the motion. Dividing $\mathbf{v}$ by its own length, $\mathbf{v}/v$, gives exactly that: it keeps the direction of the velocity but has length one. So the drag force is

$$\mathbf{F}_r = -C v^2 \frac{\mathbf{v}}{v} = -C v \mathbf{v}, \quad v = |\mathbf{v}| = \sqrt{v_x^2 + v_y^2}, \quad (8)$$

where the minus sign flips the direction, and one factor of v cancels against the denominator.

Substituting this into the component equations above,

$$a_x = -\frac{C}{m} v v_x, \quad a_y = -g - \frac{C}{m} v v_y. \quad (9)$$

2 System of Differential Equations

The equations for a_x and a_y derived above are completed by the definitions of velocity and acceleration,

$$\mathbf{v} = \frac{d\mathbf{r}}{dt}, \quad \mathbf{a} = \frac{d\mathbf{v}}{dt} = \frac{d^2\mathbf{r}}{dt^2}, \quad (10)$$

together with the initial condition

$$\mathbf{v}(t=0) = \mathbf{v}_0, \quad (11)$$

where $\mathbf{v}_0$ is a given constant.

Written out, this gives the following system of differential equations:

$$\frac{dv_x}{dt} = -\frac{C}{m} v v_x, \quad \frac{dv_y}{dt} = -g - \frac{C}{m} v v_y, \quad (12)$$

$$\frac{dx}{dt} = v_x, \quad \frac{dy}{dt} = v_y. \quad (13)$$

In principle, this system cannot be solved analytically. This is due to the nonlinear coupling introduced by $v = \sqrt{v_x^2 + v_y^2}$: unlike the linear drag case, where the equation for v_x depends only on v_x and can be solved on its own, here the equation for v_x also depends on v_y (and vice versa), so the two cannot be solved independently.

3 Numerical Integration

We have just shown that this system of differential equations cannot be solved analytically. This does not mean the motion is unknown – only that we cannot write $\mathbf{v}(t)$ and $\mathbf{r}(t)$ as a closed formula. Instead, we approximate them numerically, by advancing the motion one small step at a time rather than jumping straight to the answer.

One way to do this is the following. We split the flight of the projectile, of total duration τ , into a large number of steps of small duration Δt , say $\Delta t/\tau \leq 10^{-3}$, with τ estimated from the case without air resistance. We then assume that, within each such short step, $\mathbf{v}$ and $\mathbf{a}$ do not change appreciably, i.e. they are approximately constant.

This assumption lets us reuse the constant-acceleration formulas from introductory kinematics,

$$v = v_0 + at, \quad x = x_0 + v_0 t + \frac{1}{2}at^2, \quad (14)$$

but only over one short step at a time, since here $\mathbf{a}$ is not truly constant – it depends on the velocity, which keeps changing. Concretely, this comes from the definition of acceleration, $\mathbf{a} = d\mathbf{v}/dt$, which for a small but finite Δt gives

$$\mathbf{a} \approx \frac{\mathbf{v}(t + \Delta t) - \mathbf{v}(t)}{\Delta t} \implies \mathbf{v}(t + \Delta t) \approx \mathbf{v}(t) + \mathbf{a}(t) \Delta t. \quad (15)$$

Writing $\mathbf{v}_n$ for $\mathbf{v}(t_n)$, and $\mathbf{a}_n$ for the acceleration evaluated using the velocity at step n (from Eq. (9)), this is

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t, \quad \mathbf{a}_n = \left(-\frac{C}{m} v_n v_{n,x}, -g - \frac{C}{m} v_n v_{n,y} \right). \quad (16)$$

In other words, we repeatedly apply the ordinary constant-acceleration formula, but only for one short step, and we recompute $\mathbf{a}_n$ from the updated velocity before taking the next step.

Similarly, for the position, the second formula in Eq. (14) gives

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t + \frac{1}{2} \mathbf{a}_n (\Delta t)^2. \quad (17)$$

The last term is negligible compared to the first as $\Delta t \rightarrow 0$. To see why, compare the two terms for a typical speed $v \sim 20$ m/s, acceleration $a \sim 10$ m/s², and $\Delta t = 10^{-3}$ s:

$$v \Delta t \sim 2 \times 10^{-2} \text{ m}, \quad \frac{1}{2} a (\Delta t)^2 \sim 5 \times 10^{-6} \text{ m}. \quad (18)$$

The quadratic term is smaller than the linear one by a factor on the order of Δt itself, since it carries an extra power of Δt ; for $\Delta t \ll 1$, squaring makes it far smaller still. This is why the term is said to be of order $O[(\Delta t)^2]$: it shrinks much faster than the linear term as Δt is made smaller, and can safely be dropped, leaving

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t. \quad (19)$$

4 Stopping Condition

The scheme above advances the projectile step by step for as long as we let it. What it does not tell us is *when to stop*. The projectile has landed when its height returns to zero, and the horizontal distance covered at that moment is the range, d . We therefore need a test, applied at every step, that tells us whether the ground has been reached.

4.1 A test based on the vertical velocity

One natural test comes from the case without air resistance. There the vertical velocity is simply

$$v_y(t) = v_{0,y} - gt, \quad (20)$$

which starts positive as the projectile climbs, passes through zero at the top, and turns negative on the way down. The trajectory is a parabola, symmetric about its peak, so the projectile comes back down exactly as fast as it went up: at the moment it lands, v_y has reached $-v_{0,y}$. This gives a possible stopping test,

$$(v_y)_{n+1} \leq -v_{0,y}, \quad (21)$$

which is convenient, since v_y is already computed at every step.

4.2 What drag does to this test

Drag opposes the motion at all times, so it brakes the projectile on the way up *and* on the way down. The up-down symmetry of the parabola is therefore lost. Because drag continuously removes energy from the motion and never returns any, the projectile arrives at the ground moving more slowly than it left it,

$$|v_{y,\text{landing}}| < v_{0,y} \quad \text{whenever } C > 0, \quad (22)$$

with equality only in the drag-free case.

This means Eq. (21) is not met at the instant of landing. It may still be met slightly *after* it: below the ground the projectile keeps speeding up, and $|v_y|$ may yet reach $v_{0,y}$. Whether it does depends on the terminal velocity $v_t = \sqrt{mg/C}$, the largest downward speed drag permits:

- if $v_t > v_{0,y}$ (weak drag), the test is met a little below ground, and the simulation stops slightly late;
- if $v_t \leq v_{0,y}$ (strong drag), it is never met, and the simulation does not stop on its own.

In the first case the overshoot is small, and this is also the regime where the 45° result is recovered. The overshoot grows with C , however, and every step taken below ground still adds to x , so the range comes out slightly too large.

4.3 A test based on the height

The most direct test is to ask the question itself: the projectile has landed when its height is no longer positive,

$$y_{n+1} \leq 0. \quad (23)$$

4.4 Comparing the two

Rather than choose between them, we implement both and let the simulation measure the difference. The loop stops when

$$\underbrace{(v_y)_{n+1} \leq -v_{0,y}}_{\text{velocity test}} \quad \text{or} \quad \underbrace{y_{n+1} \leq 0}_{\text{height test}} \quad \text{or} \quad \underbrace{n > n_{\max}}_{\text{step limit}}, \quad (24)$$

recording which one triggered. The step limit is required because, as shown above, the velocity test alone may never be met.

Running both across a range of C then measures the discrepancy rather than assuming it. We expect the relative difference in range, $(d_{v_y} - d_y)/d_y$, to vanish as $C \rightarrow 0$ and to grow with

C ; where it becomes appreciable is precisely where the two tests stop agreeing. It is also worth checking whether the optimal angle θ is affected at all, since a bias in d does not necessarily shift the angle that maximises it.

Once the loop stops, the range is $d = x_{n+1}$. Since the final step generally carries the projectile just past $y = 0$, a more accurate value is obtained by interpolating linearly between the last point above the ground and the first point below it.

5 How this turns into Code

This section outlines how the equations above map onto a program. It is intentionally abstracted: the purpose is to define the structure and the reasoning behind it, not the syntax. For the simulation itself, the language chosen was C++, but the same structure could be implemented in any language.

5.1 Reducing the parameters

Before writing anything, it is worth noticing that C and m never appear separately in the equations of motion, only ever in the combination C/m . Defining

$$k \equiv \frac{C}{m}, \quad (25)$$

the mass drops out of the dynamics, and the accelerations become

$$a_x = -k v v_x, \quad a_y = -g - k v v_y. \quad (26)$$

The terminal velocity also simplifies to $v_t = \sqrt{g/k}$.

The value of k is not arbitrary. Comparing $F_r = C v^2$ with the standard drag equation $F_r = \frac{1}{2} \rho C_d A v^2$ gives

$$C = \frac{1}{2} \rho C_d A, \quad \text{so} \quad k = \frac{\rho C_d A}{2m}, \quad (27)$$

where ρ is the density of air, C_d a dimensionless shape-dependent coefficient, and A the frontal area. A program can therefore expose k at three levels: as a named preset, as a single number, or as the individual quantities ρ , C_d , A and m from which it is computed.

For interpreting results, however, neither C nor k is the natural parameter, since neither tells us how strong the drag is compared to gravity. That comparison is made by the dimensionless group

$$\beta \equiv \frac{k v_0^2}{g} = \left(\frac{v_0}{v_t} \right)^2, \quad (28)$$

To see that β is dimensionless, recall that k has units of inverse length, $[k] = \text{m}^{-1}$, since it multiplies $v v_x$ (units m^2/s^2) to produce an acceleration (units m/s^2). Then

$$[\beta] = [k] \frac{[v_0]^2}{[g]} = \text{m}^{-1} \cdot \frac{\text{m}^2/\text{s}^2}{\text{m}/\text{s}^2} = \text{m}^{-1} \cdot \text{m} = 1, \quad (29)$$

confirming that β is a pure number.

5.2 Structure

The natural decomposition follows the derivation itself. Each layer uses only the one below it:

- **Acceleration:** given a state (x, y, v_x, v_y) and k , return (a_x, a_y) . This is based on Eq. (26).
- **Step:** given a state, Δt and k , return the state one step later.

- **Trajectory:** given launch conditions, step until a stopping test fires; return the range and which test fired.
- **Sweeps:** run trajectories over a grid of θ to find the optimal angle, and over a grid of β to see how that angle varies.

Each of these is a pure function: its output depends only on its inputs. This keeps them independently testable and makes the parallelism in the next section trivially safe.

Two structural points are worth fixing early. First, the physics core should not depend on any graphics library; rendering may depend on the core, never the reverse. This allows the same code to run headless for large sweeps and interactively for exploration. Second, because the innermost functions are called millions of times, they should be defined in headers so the compiler can inline them; the layering above is for human readers, and should cost nothing at runtime.

5.3 Choosing Δt

The criterion $\Delta t/\tau \leq 10^{-3}$ requires an estimate of the flight time τ , taken from the drag-free case,

$$\tau \approx \frac{2v_0 \sin \theta}{g}. \quad (30)$$

Since τ depends on θ , so does the appropriate Δt . Deriving it per trajectory gives every angle a comparable number of steps and a comparable relative error, whereas a single global Δt leaves shallow angles over-resolved and steep ones under-resolved. When Δt is instead supplied directly, the program should check the criterion and warn when it is violated, which will happen for near-horizontal launches, where $\tau \rightarrow 0$.

5.4 Performance

Two decisions dominate, and both are structural.

The first is to avoid storing trajectories during sweeps. Only the final range is needed, so the entire state is four numbers and no memory needs to be allocated at all. Only the interactive mode requires the full path, and it draws only a handful of trajectories.

The second is where to parallelise. The Euler loop is inherently sequential, each step depends on the previous one, and must not be split. The sweeps, by contrast, consist of completely independent trajectories, one per (β, θ) pair. Flattening that grid into a single loop and distributing it across threads is safe precisely because the trajectory function is pure.

5.5 Comparing the stopping tests

The stopping test does not affect the dynamics; it only decides when we stop reading the result. A single trajectory can therefore record the step at which *each* test would have fired, and report both ranges. This halves the work relative to simulating twice, and guarantees the two are compared on the same trajectory rather than on two separately computed ones.

Whether the run ends at the first or the last test to fire can then be a user-facing option, letting the same simulation serve both as the result and as the measurement of the discrepancy discussed in Section 4.

5.6 Validation

Two independent checks are available, and both should be run before any result is trusted.

The first is physical: as $\beta \rightarrow 0$, the optimal angle must approach 45° . This tests whether the model has been implemented correctly.

The second is numerical: running the same case with Δt , $\Delta t/2$ and $\Delta t/4$ should show the answer settling down. Since Euler’s method is first order, the change between successive refinements should itself halve each time. If it does not, the integrator is wrong, regardless of how plausible the trajectories look. This plays the same role that a known analytical limit plays for the physics.

5.7 Visualisation

The results that matter here are curves rather than motion: the range as a function of launch angle, $d(\theta)$, and the optimal angle as a function of drag, $\theta_{\text{opt}}(\beta)$. For a written report these are best produced from exported data.

There is, however, one genuinely interactive element. Although a single trajectory is static once computed, the *family* of trajectories is not: varying β continuously deforms the whole fan of paths and slides the optimal angle away from 45° . An interactive view showing a fan of trajectories at different launch angles, with the optimal one highlighted, alongside a live plot of $d(\theta)$, makes the central result of this project directly visible. Recomputing a few dozen trajectories per frame is negligible work, so this costs nothing in performance.

Both modes should share the same core, differing only in what they do with the results: write them to a file, or draw them.

References

- [1] M. V. Lubarda, V. A. Lubarda, *A review of the analysis of wind-influenced projectile motion in the presence of linear and nonlinear drag force*, Archive of Applied Mechanics **92** (2022) 1997–2017.
- [2] P. S. Chudinov, V. A. Eltyshev, Y. A. Barykin, *Simple analytical description of projectile motion in a medium with quadratic drag force*, Latin American Journal of Physics Education **7** (2013).